DBT- C-DAC Mumbai
Day03

Constraints: Constraints: NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT, CHECK

A constraint is a rule applied to a table or column to control what data can be inserted or updated. Constraints help maintain data accuracy and consistency.

NOT NULL

Purpose: Prevents a column from storing NULL values

UNIQUE

Purpose: Prevents duplicate values in a column or combination of columns. In MySQL, a UNIQUE constraint allows multiple NULL values in a nullable column.

PRIMARY KEY

Purpose: Uniquely identifies every row in a table.

A primary key combines UNIQUE and NOT NULL.

DEFAULT

Purpose: Supplies a predefined value when an INSERT statement does not provide a value for that column.

CHECK

Purpose: Ensures that a value satisfies a specified condition.

MySQL began enforcing CHECK constraints in MySQL 8.0.16. Earlier versions parsed and ignored them.

- Data integrity while modifying: constraint violations, foreign key errors on insert/delete

Data integrity means maintaining the accuracy and consistency of data in a database. When we insert, update, or delete records, MySQL checks the applicable constraints to prevent invalid data.

Case A: Duplicate Primary Key

Case B: NOT NULL violation

Case C: UNIQUE violation

Case D: CHECK violation

Constraint violations during UPDATE

Constraints are checked during updates, too. A value that was valid previously may become invalid after modification.

Example A: Updating a Primary Key to a duplicate value

Example B: Updating a UNIQUE column to a duplicate value

Example C: Updating a CHECK column to an invalid value

3. Foreign key errors during INSERT

A foreign key ensures that a child record references an existing parent record.

First, create the parent table:

```
CREATE TABLE course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

INSERT INTO course VALUES
(1, 'Java'),
```

(2, 'MySQL');

```
CREATE TABLE student_course (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100),
    course_id INT,
    CONSTRAINT fk_student_course
        FOREIGN KEY (course_id)
        REFERENCES course(course_id)
);
```

Valid insert

```
INSERT INTO student_course
VALUES (101, 'Rahul', 1);
```

Invalid insert in the child table

```
INSERT INTO student_course
VALUES (102, 'Priya', 10);
```

What happens when adding constraints to existing data?

Suppose a table already contains duplicate email addresses:

Now try adding a UNIQUE constraint:

This fails because the existing records violate the new constraint.

Similarly, adding a foreign key may fail if existing child records reference parent IDs that do not exist.

Before adding a constraint, validate and correct existing data. Do not remove the constraint simply to make invalid data operations succeed.

Keys:
A key is a column or combination of columns used to identify records uniquely or establish relationships between tables. We will use a passport registration example throughout this document.

Columns: ID, aadhaarNo, panNo, name, emailid.

Assumption for this learning example: ID, Aadhaar number, PAN number and email address are unique and mandatory. In a real application, enforce uniqueness only where the business rules support it.

```
CREATE TABLE passport_registration (
        ID INT,
        aadhaarNo VARCHAR(12),
        panNo VARCHAR(10),
        name VARCHAR(100),
        emailid VARCHAR(100)
);
```

```
CREATE TABLE passport_application_details (
        applicationNo INT PRIMARY KEY,
         RegistrationID INT,
        applicationDate DATE,
        status VARCHAR(30),
        FOREIGN KEY (RegistrationID) REFERENCES passport_registration(ID)
);
```

3. Super Key

A Super Key is any column or combination of columns that can uniquely identify a row. A super key may contain unnecessary attributes.

Possible examples under the assumptions above:

- ID
- aadhaarNo
- panNo
- emailid

- ID + name
- aadhaarNo + name
- ID + aadhaarNo + name

MySQL does not provide a separate SUPER KEY constraint; super key is a database-design concept.

Candidate Key

A Candidate Key is a minimal super key. It uniquely identifies a row, and removing any attribute from it would mean it no longer uniquely identifies the row.

Candidate keys in this example: ID, aadhaarNo, panNo and emailid, assuming each is unique and required.

ID + name is not a candidate key because ID alone is enough.

Primary Key

A Primary Key is the candidate key selected as the main identifier for records. In this example, ID is selected.

- Duplicate values are not allowed.
- NULL values are not allowed.
- A table can have only one PRIMARY KEY constraint; that key may contain one or multiple columns.
- MySQL automatically creates a unique index for the primary key.
- A primary key identifies records and can be referenced by foreign keys.
- It is recommended that each table has a primary key.

TEXT and BLOB columns cannot be used as unrestricted full-column primary keys in MySQL. Indexing them has prefix-length restrictions; ordinary numeric or VARCHAR identifiers are more common.

ALTER TABLE passport_registration

ADD CONSTRAINT pk_passport_registration PRIMARY KEY (Id);

Alter table passport_registration modify Id Int primary key;

Alter table passport_registration add primary key (id);

Alternate Key

An Alternate Key is a candidate key that was not selected as the primary key. If ID is the primary key, aadhaarNo, panNo and emailid are alternate keys in this example.

MySQL has no separate ALTERNATE KEY keyword. UNIQUE constraints enforce uniqueness for these columns.

Composite Primary Key

A Composite Primary Key combines two or more columns to uniquely identify a row. It is useful when no single column in the combination is sufficient on its own.

- The combination must be unique.
- Individual columns can contain duplicates.
- MySQL creates a composite index for the composite primary key.
- MySQL supports up to 32 key parts in a multi-column index, subject to other limits.

Surrogate Key

A Surrogate Key is an artificial identifier created by the system instead of using a real-world attribute as the primary key. A numeric AUTO_INCREMENT ID is a common choice.

- The database generates the ID automatically.
- The key does not depend on a real-world identifier.
- It is usually compact and remains stable if email or other personal details change.
- INT AUTO_INCREMENT is common for numeric surrogate keys. CHAR can be used for fixed-length generated codes, but is not inherently recommended for every surrogate key.

Foreign Key

A Foreign Key connects tables by referencing a primary key or eligible unique key in another table. It maintains referential integrity.

Here, passport_person.ID is the primary key and passport_application_details.personID is the foreign key. An application cannot refer to a person ID that does not exist in the parent table. A nullable foreign-key column may be NULL unless another constraint prohibits it.

- Aggregate functions: COUNT(*) vs COUNT(col), SUM, AVG, MIN, MAX; NULL handling

Aggregate functions perform calculations on multiple rows and return a single result for the selected group of rows.

For example, they can calculate the total marks, average salary, maximum price, or number of students.

Remember: COUNT(*) counts rows; COUNT(column) counts non-NULL values.

SUM()

Calculates the total of the non-NULL numeric values.

AVG()

Calculates the average of the non-NULL numeric values.

MIN()

Returns the smallest non-NULL value.

MAX()

Returns the largest non-NULL value.

Both MIN() and MAX() can also work with dates and strings, according to their ordering rules.

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(marks) AS marks_count,
    SUM(marks) AS total_marks,
```

```
    AVG(marks) AS average_marks,
    MIN(marks) AS minimum_marks,
    MAX(marks) AS maximum_marks
FROM student;
```

Aggregate function with GROUP BY

```
SELECT city, AVG(marks) AS average_marks
FROM student
GROUP BY city;
```

- GROUP BY: single and multiple columns

What is GROUP BY?

The GROUP BY clause in MySQL is used to group rows that have the same values in one or more columns.

It is commonly used with aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX() to calculate results for each group.

Example: If a student table contains students from Pune and Mumbai, GROUP BY city groups students city-wise.

Important rules to remember

- GROUP BY groups rows with matching values.
- It is commonly used with aggregate functions.
- It can group by one or multiple columns.
- NULL values in a grouping column are placed in the same group.
- GROUP BY does not guarantee the order of the output. Use ORDER BY when sorting is required.

- When using aggregate functions with other selected columns, those non-aggregated columns generally need to be included in GROUP BY or be functionally dependent on the grouped columns under MySQL's ONLY_FULL_GROUP_BY rules.

  If you select a non-aggregated column that is not in GROUP BY, MySQL generally rejects the query unless that column is functionally dependent on the grouped columns.

Distinct : How to use DISTINCT

MySQL: WHERE vs HAVING — Theory Summary

1. WHERE Clause

The WHERE clause is used to filter individual rows before grouping or aggregation takes place.

HAVING Clause

The HAVING clause is mainly used to filter groups based on conditions, especially conditions involving aggregate functions such as COUNT(), SUM(), and AVG().

3. Difference between WHERE and HAVING

WHERE	HAVING
Filters individual rows.	Commonly filters groups or aggregate results.

Applied before grouping.	Applied after grouping.
Commonly used with normal column conditions.	Commonly used with aggregate conditions.
Cannot directly use aggregate functions in its condition.	Can use aggregate functions in its condition.
Can be used without GROUP BY.	Can also be used without GROUP BY.

Can HAVING work like WHERE?

Yes. MySQL allows HAVING to filter rows even without GROUP BY or an aggregate function.

```
SELECT city
FROM student
HAVING city = 'Pune';
```

```
SELECT city
FROM student
WHERE city = 'Pune';
```

However, use WHERE for ordinary row filtering. Using HAVING this way is generally unnecessary.

Logical order of query execution

The simplified logical execution order of a typical MySQL query is:

1. FROM — identifies the source table.
2. WHERE — filters individual rows.
3. GROUP BY — groups matching rows.
4. HAVING — filters groups or aggregate results.

5. ORDER BY — sorts the result.
6. LIMIT — restricts the number of rows returned.

```
create table passport_application
( app_no int primary key,
application_date date,
status varchar(15),
registration_id int,

Foreign key (registration_id) references passport_registration(id)
On delete cascade
On update cascade
);
```

```
create table passport_application
( app_no int primary key,
application_date date,
status varchar(15),
registration_id int,
```

```
Constraint fk_passport_application_reg

Foreign key (registration_id) references passport_registration(id)

On delete set null

On update set null

);


alter table passport_application

add constraint fk_passport_application_reg

Foreign key (registration_id) references passport_registration(id)

On delete restrict

On update restrict;
```

ON DELETE CASCADE

Meaning: Delete the related child records automatically when the parent record is deleted.

ON DELETE SET NULL

Meaning: Keep the child records but set their foreign key to NULL when the parent is deleted.

Important: The foreign-key column must allow NULL. If it is declared NOT NULL, this action cannot work.

ON DELETE RESTRICT

Meaning: Prevent deletion of the parent record if child records refer to it.

ON UPDATE CASCADE

Meaning: Automatically update the foreign-key values in child records.

ON UPDATE SET NULL

Meaning: Set the child foreign key to NULL when the referenced parent key changes.

The child column must allow NULL.

ON UPDATE RESTRICT

Meaning: Prevent changes to the parent key while child records reference it.

- LIKE with % and _, DISTINCT
- ORDER BY: ASC/DESC, multiple columns; LIMIT and OFFSET
- BETWEEN ... AND, IN / NOT IN, IS NULL / IS NOT NULL (why = NULL fails)

Between ….AND

The BETWEEN operator selects values within a specified range. Both the starting and ending values are included in the range

IN operator

The IN operator checks whether a value matches any value in a specified list.

NOT IN operator

The NOT IN operator selects rows where a value does not match any value in the specified list.

IS NULL and IS NOT NULL

In SQL, NULL represents a missing or unknown value. It is not the same as zero, an empty string, or a blank space.

IS NULL

Used to find rows where a column has a NULL value.

IS NOT NULL

Used to find rows where a column contains a non-NULL value.

Why does = NULL fail?

The condition evaluates to unknown, not true or false, because SQL does not treat NULL as an ordinary value.

Why does SQL behave this way?

Suppose a student's marks are NULL. We do not know the actual marks, so SQL cannot determine whether those marks equal NULL or differ from it. Therefore, ordinary comparison operators such as =, <>, and != are not suitable for testing NULL.